

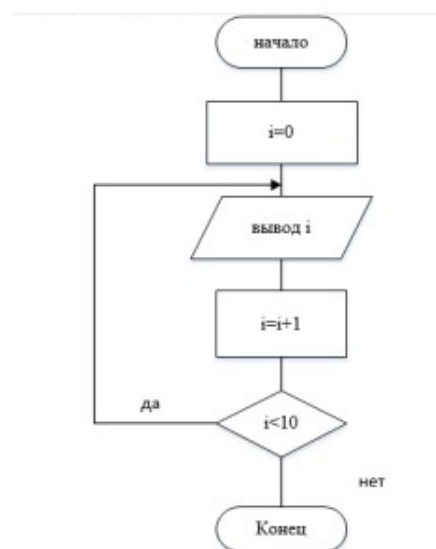
Лабораторная работа 6

Циклы с заданным числом повторений

Предмет исследований. Организация циклов с известным числом повторений. Инструкция циклов `for`. Разработать алгоритмы решения задач. Составить программы решения задач.

Задание 1. Вывести на экран числа от 0 до 9 (вместо 10 можно взять число, равное двум последним цифрам номера зачётной книжки).

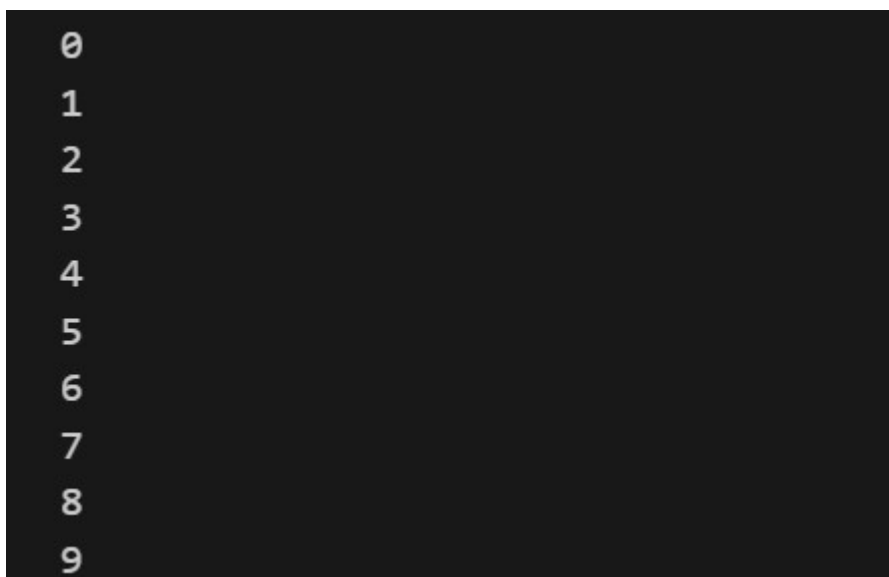
Блок схема



Блок схема для задания 1.

Пример решения задачи в виде блок-схемы (описание)

1. **Начало.**
2. **Инициализация счётчика:** $i = 0$.
3. **Вывод значения i .**
4. **Увеличение счётчика:** $i = i + 1$.
5. **Проверка условия:** $i < 10$?
 - да → переход к шагу 3;
 - нет → переход к шагу 6.
6. **Конец.**



Результат задания 1.

Задание 2. Вычислить значения функции с использованием инструкции цикла for.

№	Задача 1	Задача 2
	$a = 2.8; x = 5.1; c = 1.8; b = 0.81$	
1.	$t = \sum_{n=2}^5 \cos^2(na)$	$t = \sum_{n=1}^{10} \prod_{a=1}^{10} \cos(na)$
2.	$z = \prod_{k=1}^{12} (\cos(k) + \sin(k))$	$t = \prod_{a=1}^5 \sum_{b=2}^{10} \sin(ab)$
3.	$p = \prod_{n=1}^{15} \cos(nbcx)$	$t = \prod_{b=1}^{10} \sum_{c=2}^{10} \lg(bc)$
4.	$z = \sum_{k=1}^{18} (\cos(ak) + \sin(c))$	$t = \prod_{a=1}^5 \sum_{c=1}^{10} \ln(ac)$
5.	$z = \prod_{n=1}^{15} (a^2 + \ln(nc))$	$t = \prod_{a=2}^{10} \sum_{b=1}^{10} \cos(a+b)$
6.	$z = b \cdot \prod_{n=1}^{13} \cos(n^2b)$	$t = \prod_{b=2}^5 \sum_{c=1}^{10} bc$
7.	$z = \prod_{n=1}^{40} (\cos(an) + bn)$	$t = \prod_{a=1}^5 \sum_{c=1}^3 (a^2 \sqrt{c})$
8.	$t = \sum_{n=1}^{12} (x \cdot \sin(na))$	$t = \prod_{b=2}^8 \sum_{c=1}^{10} (\sqrt{b} + c)$
9.	$u = \sum_{n=1}^{17} (n \cdot \sin^2(n^2a))$	$t = \prod_{a=2}^5 \prod_{b=1}^5 b^2 a$
10.	$t = \sum_{n=2}^{10} 5 \cdot \lg(abn)$	$t = \prod_{c=2}^7 \sum_{b=1}^5 cb^2$

Варианты для заданий 2 – 3.

Блок схема



Блок схема в общем виде.

Вложенные циклы for

Задание 3. Вычислить значения функции с использованием вложенных инструкций цикла for. Вариант задания в таблице Проект консольное приложение. Пример данного задания рассмотрен в лекции.

Команда break

Задание 4. Создать программу с использованием команды break.

Проект – консольное приложение. В программе ищется первое не простое число из последовательности чисел от N до 1. Варианты заданий – N = две последние цифры номера зачетной книжки.

Команда continue

Задание 5. Создать программу с использованием команды continue.

Проект – консольное приложение. В программе ищется первое нечетное число из последовательности чисел от N до 1. Варианты заданий – N = две последние цифры номера зачетной книжки.

Методические указания

1. Цикл **for** и функция **range**.

В Python цикл по счётчику организуется так:

```
for i in range(start, stop, step):
```

```
    # тело цикла
```

Значения *i* пробегают от *start* до *stop* - 1 с шагом *step*.

Для простых циклов часто достаточно `for i in range(n):` (от 0 до *n*-1).

2. Переменная-накопитель.

Для суммирования и перемножения заранее вводится переменная:

```
s = 0.0    # для суммы
```

```
p = 1.0    # для произведения
```

Внутри цикла значение обновляется: `s += выражение` или `p *= выражение`.

3. Использование модуля **math**.

Для тригонометрических и логарифмических функций необходимо:

```
import math
```

Основные функции: `math.sin(x)`, `math.cos(x)`, `math.tan(x)`,

`math.log(x)` (натуральный логарифм), `math.sqrt(x)` (квадратный корень).

4. Вложенные циклы.

Если в формуле присутствуют двойные суммы или произведения (по двум индексам), используют вложенный цикл:

```
result = 0.0
```

```
for i in range(...):
```

```
    for j in range(...):
```

```
        result += выражение_от_i_j
```

5. Оператор **break**.

`break` немедленно завершает выполнение текущего цикла.

Используется для досрочного выхода при выполнении некоторого условия (например, найдено нужное число).

6. Оператор **continue**.

continue пропускает оставшуюся часть тела цикла и переходит к следующей итерации.

Подходит для пропуска «лишних» элементов (например, всех чётных чисел при поиске нечётных).

7. Проверка и тестирование.

Для простых формул полезно проверить результат вручную (на бумаге или в калькуляторе) для небольших диапазонов, чтобы убедиться в правильности работы программы.

Требования к отчету

1. Титульный лист
2. Задание.
3. Блок схему
4. Листинг программы
5. Результат выполнения программы.

Контрольные вопросы

1. Преимущества использования инструкций циклов в программе.
2. Инструкция цикла for.
3. Как организовать цикл с нарастанием индекса?
4. Как организовать цикл с убыванием индекса?
5. Организация вычисления суммы.
6. Организация вычисления произведения.

Лабораторная работа № 1 — Численные типы в языке Python

Предмет исследований: Структура программы на языке Python. Задание констант и переменных. Типы численных данных и ошибки при преобразованиях типов. Организация простейшего ввода-вывода данных. Программа решения задачи в виде консольного приложения.

Задание

Создать консольную программу взаимных преобразований численных типов данных в соответствии с вариантом. В программе должны быть:

- Преобразования типа `int` в другие целочисленные типы.
- Взаимные преобразования типа `int` и типов с плавающей точкой.
- Преобразование типа `float` в `int` и обратно.

При преобразованиях нужно проверять получаемые значения и размер в байтах (с помощью `sys.getsizeof()`). Данные в консоль выводятся функцией `print()`. Для форматного вывода можно использовать **f-строки**.

Варианты к заданию

№	Значение вещественного числа	Значение целого числа
1	123.45	150
2	23.456	160
3	34.567	170
4	456.78	180
5	567.89	190
6	67.890	200
7	78.901	210
8	890.12	220
9	91.012	230
10	109.876	240

Преобразуем int в другие целые типы

Типы	int	Int16	Int32	Int64	Byte	sbyte
Значения	132	132	132	132	132	-124
Байты	28	2	4	8	1	1

Преобразуем int в типы с плавающей точкой

Типы	int	double	Single
Значения	225	225.0	225.0
Байты	28	8	4

Преобразуем double в Single и int

Типы	double	Single	int
Значения	184.857	184.857	184
Байты	8	4	28

Рис 1.1 – Результат выполнения работы

Требования к отчёту

Отчёт подготавливается в электронном виде. Он должен содержать постановку задачи, скриншоты, которые показывают порядок решения задачи, необходимые комментарии и анализ полученных результатов. И электронное решение задач.

Контрольные вопросы:

1. Алфавит языка Python, операции, идентификаторы.
2. Структура программы на Python.
3. Переменные. Их объявление.
4. Форматы представления чисел (целые и с плавающей точкой).
5. Типы целых чисел в Python: int, bool.
6. Типы вещественных чисел (с плавающей точкой): float, complex.
7. Преобразования типов при присвоениях.
8. Получение размера объектов с помощью sys.getsizeof().

Лабораторная работа № 3 — Линейные структуры

Предмет исследований: Запись констант, переменных, стандартных функций. Правила записи арифметических выражений. Арифметические операторы присваивания. Разработка алгоритма решения в соответствии с заданием. Составление программы решения задачи в виде консольного приложения.

Задание. Вычислить значения переменных в соответствии с вариантами задания. Вывести значения вводимых исходных данных и результаты, сопровождая их вывод именами выводимых переменных. Задание выполнить в виде консольного приложения. Номер варианта определяется номером ЭВМ в аудитории (закрепляется на первом занятии на весь семестр).

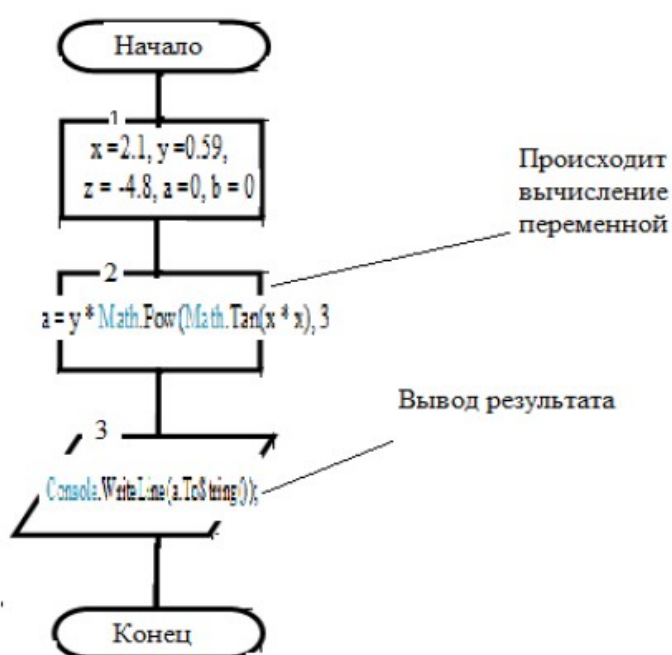
Ввод данных для вычислений должен производиться с клавиатуры.

Работу программы проверить при различных наборах исходных данных не менее 6 раз. Вводимые данные для вычислений должны содержать положительные и отрицательные вещественные и целые числа.

№	Задание	№	Задание
1	$-10 * \frac{\cos^{10}(e^{x^2})}{\operatorname{ctg}^{10}(\cos^{10}(x))} * \frac{5^{x^{(10^6)}}}{\sin^{10}(x^6)}$	8	$\frac{\operatorname{ctg}^3(x^2 - z)}{-e^{yx}} + \cos^2(a^4 - b) $
2	$\left(\left(\frac{\cos^5\left(\frac{\pi}{3}\right)}{\operatorname{ctg}^5\left(\frac{\pi}{3}\right)} + \left(\frac{\sin^3\left(\frac{\pi}{9}\right)}{\operatorname{ctg}^3\left(\frac{\pi}{9}\right)} \right)^a \right)^a$	9	$\frac{\cos^4(t^2 - z)}{\ln(20 - a)} + \frac{\sqrt{x^3 + 10}}{\sin^2(5^n + m)}$
3	$\frac{(abc + \operatorname{tg}^5 x)^6}{75 - 43 * x^x} * \sin^5\left(n \frac{\pi}{6}\right)$	10	$\frac{p^{x^{(x^4 + u)}}}{\sqrt{e^{x^2} + c}} + \frac{\operatorname{ctg}^4(b^3 + 10)}{\operatorname{tg}^2(a^2 + 20)}$
4	$\frac{\sqrt[4]{-8 * (-b)^n} + \cos^n(r + n)}{((-8b)^3)^4 + \operatorname{ctg}^n(r + n)}$	11	$\frac{\left(\left(\frac{e^{-xy}}{a^3 * 100} \right)^{\frac{\pi}{9}} * \frac{\cos^{x^3}(b^3 + 11)}{\operatorname{ctg}^3(y^3 + 20)} \right)^n}{e^{y * x}}$
5	$\left(\sqrt[3]{\operatorname{tg} x^{e^{a+2}}} \right)^3 \left((axn)^3 \sqrt{\operatorname{ctg}(x^{e^{a+2}})} \right)^3$	12	$\frac{\sqrt[3]{e^{x * y} + \cos^2(a^{10} + b^{10})}}{\operatorname{ctg}^2\left(n \frac{\pi}{100}\right) * \cos^4\left(\frac{\pi}{3} n\right)} + \sin^4(2x)$

6	$\frac{\cos^x(n*a*b)^{e^{7n}}}{-1*\sqrt{-1*a(b*\sin^x(n*a*b)^{e^{7n}}}}}$	13	$\frac{ ctg^3 n^{18} }{\sqrt{ e^{n^{a+n}} }} - 1$
7	$\left(\frac{e^{x^{(b^4+4)}}}{ctg^2\left(\frac{\pi}{12}*x\right)} * \frac{ \ln(8x^a + b^{10}) }{\ln(5x^{10} + a^2)} \right)$	14	$\frac{-1*\left\ \cos^n e^{x^n} \right\ * (-1)*r}{\left(\sqrt[4]{x^n + x^r + x^x} \right)^4}$

Пример блок схемы



Внимание. При вводе данных в консоли разделитель целой и дробной части вещественного числа – запятая. А при вводе данных точка.

```

Результаты
a=21,6349964791028
b=2,17968494366062
Нажмите любую клавишу
  
```

Результат следует сократить до двух знаков после запятой.

Отчет по лабораторной работе представить преподавателю в следующем виде:

1. Титульный лист
2. Задание.
3. Блок схему
4. Листинг программы
5. Результат выполнения программы.

Контрольные вопросы

1. Алфавит языка Python.
2. Операции в Python.
3. Идентификаторы в Python.
4. Типы данных в Python.
5. Структура программы консольного приложения на Python.
6. Где описываются константы, переменные и типы данных в Python?
7. Стандартные функции в Python.
8. Операторы присваивания в Python.
9. Простые и составные инструкции.
10. Функция ввода `input()`.
11. Функция вывода `print()`.
12. Последовательность действий при выполнении оператора присваивания.
13. Приоритетность выполнения операций в выражениях.
14. Как организовать пропуск одной, двух строк при выводе?
15. Особенности работы с вещественными числами в Python.
16. Преобразование типов данных (`int()`, `float()`, `str()`).
17. Форматирование вывода чисел (f-строки, `format()`).
18. Модуль `math` и его основные функции.

Лабораторная работа № 4 — Структура ветвление

Предмет исследований: Условная и безусловная передача управления; Вычислительные процессы с разветвляющейся структурой. Разработать алгоритмы решения в соответствии с заданием. Составить программы решения задач.

Ветвление if; elif; else

Задание

Вычислить значения функции по варианту задания. Вывести значения исходных данных и полученные результаты, сопровождая их именами переменных. Значения аргумента взять из указанного диапазона так, чтобы протестировать все ветви программы. Проект – консольное приложение. Значения не определенные исходными данными запрашивать у пользователя.

Работу программы проверить при различных наборах исходных данных не менее 6 раз. Вводимые данные для вычислений должны содержать положительные и отрицательные вещественные и целые числа.

Заполнить отчет

Примечание. В Python отсутствует оператор безусловного перехода goto. Вместо этого используются структурные конструкции управления потоком выполнения: условные операторы, циклы, функции.

Варианты заданий

Номер варианта определяется номером ЭВМ в аудитории (закрепляется на первом занятии на весь семестр).

Вариант	Задание	Диапазон
1	$f = \begin{cases} tg^2(x+z), & x < -8 \\ e^x * \sin(x^3+z), & -8 \leq x < 3 \\ \ln(z-x^2) , & x \geq 3 \end{cases}$	$x = [-15, 15]; \Delta x = 3$
2	$h = \begin{cases} \ln(\sin(x^3)), & x \leq -2 \\ a^{x-a} + \cos^2(x+a), & x \geq 3 \\ e^{a*x} - tg\left(\frac{x}{(x+a)}\right), & -2 < x < 3 \end{cases}$	$x = [-3, 5]; \Delta x = 0,2$

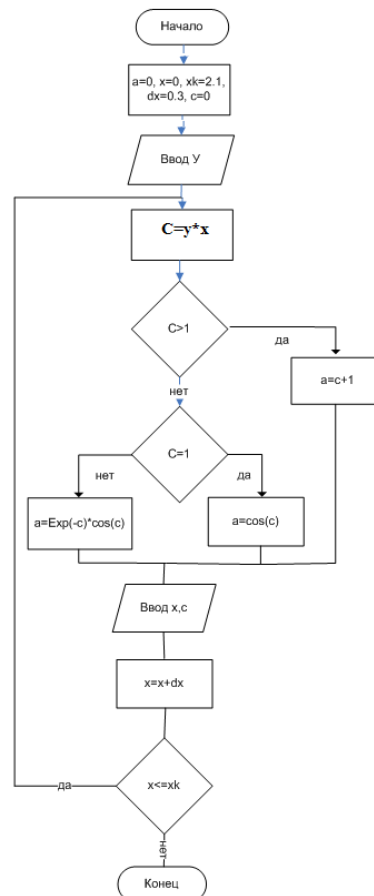
3	$a = \begin{cases} e^x * x^3 + \ln(x - z), z < 0 \\ e^{-x} \sin^3(x^2 + z), 0 \leq z < 2 \\ e^{(x-z)} \cos(x - z^3) , z \geq 2 \end{cases}$	$z = [-5, 5]; \Delta z = 1$
4	$x = \begin{cases} \cos(y + a), y \leq 0 \\ \operatorname{tg}(y - a), 0 < y \leq 10 \\ \frac{\ln(y * a)}{y^3 + a^y}, y > 10 \end{cases}$	$y = [-5, 15]; \Delta y = 1$
5	$p = \begin{cases} \frac{\ln(z - a)}{\sin^2(z + a)}, z \leq 5 \\ \frac{\operatorname{tg}(a^3 + z)}{z}, 5 < z < 10 \\ e^{(a*2+3)}, z \geq 10 \end{cases}$	$z = [0, 20]; \Delta z = 1$
6	$k = \begin{cases} \ln(\cos^3(p * a)), a \leq 5 \\ \sin^3\left(\frac{p}{(p+a)^2}\right), 5 < a \leq 10 \\ e^{(p-\frac{p}{a})}, a > 10 \end{cases}$	$a = [0, 13]; \Delta a = 0,75$
7	$x = \begin{cases} \frac{b^2}{\ln(y^3)}, y < 1 \\ \sin^2(y + b) , 1 \leq y \leq 5 \\ \frac{\cos^3(y - b)}{ y - b }, y > 5 \end{cases}$	$y = [-3, 10]; \Delta y = 2$
8	$c = \begin{cases} z^3 + \sin(3^z * z), z < -5 \\ \cos^2(z^4 - x), -5 \leq z < 7 \\ \frac{\operatorname{tg}(z^3)}{ \cos(x) }, z \geq 7 \end{cases}$	$z = [-10, 10]; \Delta z = 1,5$
9	$w = \begin{cases} \cos\left(\frac{z}{a} + z^3\right), z \leq 5 \\ a^{(z+a)} + \sin^3\left(\frac{z}{a}\right), 5 < z < 15 \\ \ln(z - a^2) , z \geq 15 \end{cases}$	$z = [0, 20]; \Delta z = 1,5$

10	$d = \begin{cases} \frac{\sin^3(z+c)}{\cos^2(z-c)}, z < 10 \\ \operatorname{tg}^3(z^{c-z}), 10 \leq z < 10 \\ e^{\frac{c-z}{c}} + \sqrt{c^2+z^2}, z \geq 20 \end{cases}$	$z = [5, 25]; \Delta z = 3$
11	$h = \begin{cases} \sin^d(d+z), d < 5 \\ d^{\cos\left(\frac{d}{z}\right)}, 5 \leq d \leq 15 \\ \ln\left(\left \cos\left(d - \frac{z}{d}\right)\right \right), d > 15 \end{cases}$	$d = [1, 26]; \Delta d = 1,5$
12	$r = \begin{cases} \frac{b^2}{\ln(b-c)}, b \leq -2 \\ \operatorname{tg}^2(b^c+c), b \geq 4 \\ \frac{b^{b+c}}{ \sin(b-c) }, -2 < b < 4 \end{cases}$	$b = [-7, 6]; \Delta b = 1$
13	$x = \begin{cases} \ln(b^{n-1}), n \leq 0 \\ e^{b(n-1)}, 0 < n < 5 \\ \frac{\sin(b*n-5)}{ \cos(b*n) }, n \geq 5 \end{cases}$	$n = [-8, 9]; \Delta n = 0,5$
14	$x = \begin{cases} \frac{\sin^3(z+a)}{\ln\left(\frac{z}{a}\right) - \cos(z)}, z \leq -3 \\ \left \sin(z^3) - \cos(z*a) \right , -3 < z < 2 \\ \left \frac{z^4-1}{\operatorname{tg}^z\left(\frac{z}{a}\right)} \right , z \geq 2 \end{cases}$	$z = [-4, 4]; \Delta z = 0,5$
дополнительно	$y = \begin{cases} \sin\left(\frac{x^3}{x+3}\right), x \leq 0 \\ \cos^3\left(\frac{x-a}{x^2}\right), 0 < x \leq 5 \\ \ln\left(\frac{a*x}{x^a}\right), x > 5 \end{cases}$	$x = [-20, 20]; \Delta x = 2$

Пример. Вычислить при $y=1.3$, $x=[0.. 2.1]$ с шагом 0.3 значения функции a .
Результат вывести в виде таблицы. Проект – консольное приложение.

$$a = \begin{cases} yx + 1 \dots n p u \dots yx > 1 \\ \cos(yx) \dots n p u \dots yx = 1 \\ e^{-yx} \cos(yx) \dots n p u \dots yx < 1 \end{cases}$$

Блок-схема алгоритма представлена на рисунке.



Для организации множественных вычислений введены следующие переменные: $\underline{x}$ - начальное значение, $\underline{xk}$ - конечное значение $\underline{dx}$ - шаг изменения аргумента $\underline{x}$. Для осуществления повторного вычисления используется проверка условия вхождения переменной $\underline{x}$ в заданный диапазон, и $yx=C$ для упрощения выражения.

Консоль перед закрытием программы:

```
ВВЕДИТЕ m
1.3
x = 0 a = 1
x = 0.3 a = 0.626216017262028
x = 0.6 a = 0.325887017343072
x = 0.9 a = 2.17
x = 1.2 a = 2.56
x = 1.5 a = 2.95
x = 1.8 a = 3.34
x = 2.1 a = 3.73
НАЖМИТЕ ЛЮБУЮ КЛАВИШУ
```

Ветвления. Самостоятельное задание.

Самостоятельно разработать задачу, включающую вложенное ветвление. Условия выбора соответствующих веток выбрать самостоятельно. Функции для веток подобрать самостоятельно, при этом в каждой ветви должно быть не менее двух функций из модуля **math**. Ввод начального, конечного значения вычисляемого диапазона, а также шаг вычисления вводятся пользователем при запуске программы. Результат выполнения выводятся в консоль в виде таблицы.

Выбор switch; case

Задание

Рассмотреть представленный пример и на основе него самостоятельно разработать программу вычисления математической функции состоящей из 5 веток. Для выбора ветки используется одна из цифр номера Вашего студенческого билета – ABCDE.

Производимые вычисления выбрать самостоятельно.

При выполнении программы в зависимости от вводимого целого положительного числа выбирается одна из веток множественного ветвления и производится вычисление с выводом результата. Если введенное значение не найдено выводится сообщение об отсутствии подходящей функции.

Контрольные вопросы

1. Какие структуры вычислительных процессов Вы знаете?
2. Как организовать разветвление вычислений в Python?

3. Ветвление if... elif... else.
4. Вложенные ветвления.
5. Инструкция выбора match case (Python 3.10+).
6. Особенности использования оператора match в Python.
7. Как организовать множественное ветвление в версиях Python до 3.10?
8. Какие модули математических функций доступны в Python?
9. Как организовать проверку принадлежности значения диапазону?
10. Особенности работы с вещественными числами в условиях.

Лабораторная работа № 5 — Циклы с неизвестным числом повторений

Предмет исследований. Организация циклов с неизвестным числом повторений. Инструкции циклов **while**. Вычисление суммы членов бесконечного ряда. Разработать алгоритмы решения задачи. Составить программы решения задачи.

Цикл **while**

Задание 1

Вычислить значение суммы членов бесконечного ряда с заданной точностью **E** с использованием инструкции цикла **while**. На печать вывести значение суммы и число членов ряда, вошедших в сумму. Проект – консольное приложение.

Варианты заданий

№	Сумма членов ряда	x	Точность
1.	$s = -\frac{(2x)^2}{2!} + \frac{(2x)^4}{4!} + (-1)^n \frac{(2x)^{2n}}{(2n)!} + \dots$	0.1	0.00001
2.	$s = x - \frac{x^3}{3} + \dots + (-1)^{n-1} \frac{x^{2n-1}}{2n-1} + \dots$	0.1	0.00005
3.	$s = \frac{x^3}{5} - \frac{x^5}{17} + (-1)^{n+1} \frac{x^{2n+1}}{4n^2+1} + \dots$	0.15	0.001
4.	$s = 1 + \frac{x}{1!} \cos \frac{\pi}{4} + \dots + \frac{x^n}{n!} \cos\left(\frac{\pi}{4} n\right) + \dots$	0.12	0.0001
5.	$ch(x) = s = 1 + \frac{x^2}{2!} + \dots + \frac{x^{2n}}{(2n)!} + \dots$	0.7	0.0001
6.	$sh(x) = s = x + \frac{x^3}{3!} + \dots + \frac{x^{2n+1}}{(2n+1)!} + \dots$	1.7	0.001
7.	$arctg(x) = s = \frac{1}{x} + \dots + (-1)^n \frac{1}{(2n+1)x^{2n+1}} + \dots$	1.5	0.0005
8.	$\pi = s = 4 \left(1 - \frac{1}{3} + \dots + (-1)^n \frac{1}{2n+1} + \dots \right)$		0.0001
9.	$\cos \frac{\pi}{6} = s = 1 - \frac{(\pi/6)^2}{2!} + \dots + (-1)^n \frac{(\pi/6)^{2n}}{(2n)!} + \dots$		0.00005
10.	$\sin \frac{\pi}{3} = s = \frac{\pi}{3} - \frac{(\pi/3)^3}{3!} + \dots + (-1)^n \frac{(\pi/3)^{2n+1}}{(2n+1)!} + \dots$		0.00005

Пример. Вычислить значение суммы членов бесконечного ряда

$$s = x - \frac{x^3}{3!} + \frac{x^5}{5!} + \dots + (-1)^n \frac{x^{2n+1}}{(2n+1)!} + \dots$$

при $x = 0.1$ с точностью до члена ряда с модулем, меньшим $E=0.00001$.

Для вычисления очередного члена ряда будем использовать рекуррентное соотношение, связывающее его с предыдущим членом $a(n+1) = q \cdot a(n)$. Применение рекуррентных формул позволяет избежать вычисления факториала и возведения в произвольную степень. Рекуррентный коэффициент q найдем из выражений для текущего и следующего членов ряда

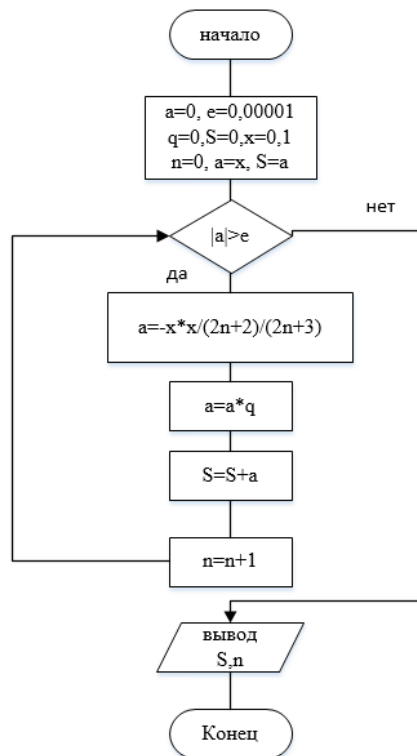
$$a(n) = (-1)^n \frac{x^{2n+1}}{(2n+1)!} \quad \text{и} \quad a(n+1) = (-1)^{n+1} \frac{x^{2(n+1)+1}}{(2(n+1)+1)!}$$

Деля второе выражение на первое, получим

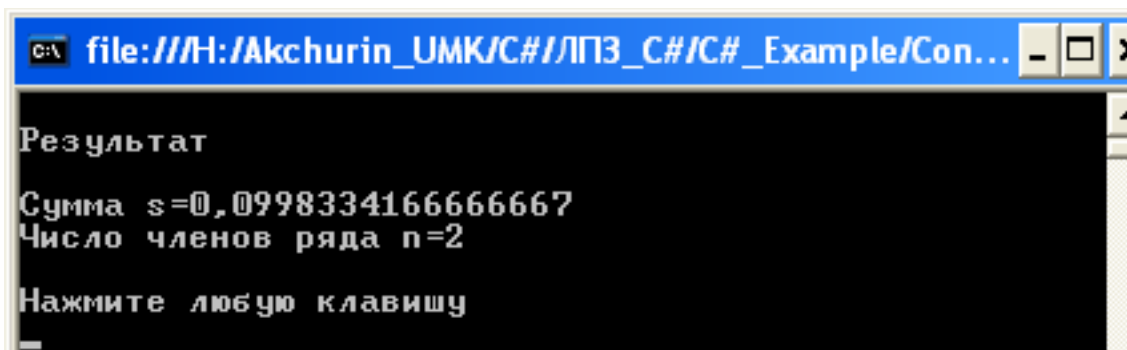
$$q = \frac{a(n+1)}{a(n)} = \frac{-x^2}{(2n+2)(2n+3)}$$

Значение начального члена ряда задаем до цикла путем прямого присваивания (номер начального члена n в разных вариантах равен 0 или 1, правильное значение определяется по формуле текущего члена). В нашем задании $n=0$, $a=x$.

Блок схема



Консоль перед закрытием программы рис.4.1

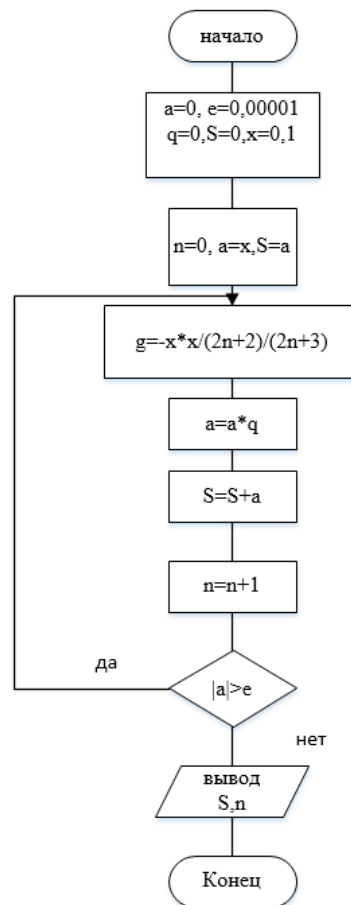


Задание 2. Ограничьте значение Суммы до 5 знаков после запятой!

Цикл с постусловием (аналог do...while)

Выполнить ту же задачу с применением конструкции, имитирующей цикл **do...while**. В Python отсутствует прямой аналог **do...while**, но его можно эмулировать с помощью **while True** с условием выхода. Проект – консольное приложение.

Блок схема



Сравните работы программ while и эмуляции do...while, объясните чем они отличаются эти процедуры и каков результат.

Контрольные вопросы

1. Циклический процесс с неизвестным числом повторений.
2. Его отличия от цикла с заданным числом повторений.
3. Инструкции языка Python для организации таких циклов.
4. Синтаксис инструкции while.
5. Как выполнить группу операторов в цикле while?
6. Как эмулировать цикл do...while в Python?
7. Синтаксис инструкции for для итерации по коллекциям.
8. Прямое вычисление суммы членов бесконечного ряда.
9. Вычисление суммы членов бесконечного ряда по рекуррентной формуле.
10. Условие выхода из цикла при вычислении суммы членов бесконечного ряда.

11. Использование операторов break и continue в циклах.
12. Организация бесконечных циклов и способы их прерывания.
13. Особенности работы с вещественными числами в циклах.
14. Контроль точности вычислений в циклических процессах.

Лабораторная работа 8 — Обработка двумерного массива

Предмет исследований. Способы описания размеров массивов в Python. Способы ввода и вывода массивов. Реализация приемов накопления суммы или произведения элементов массивов, запоминания результатов.

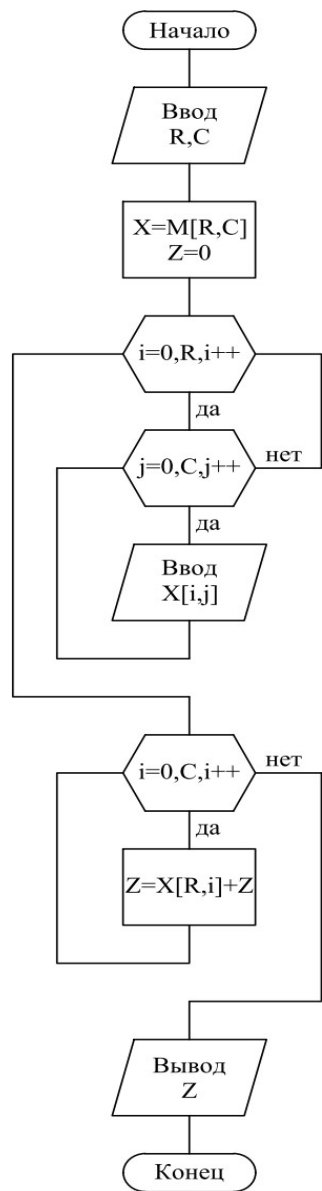
Варианты задания

№	Задание
1.	Вычислить сумму элементов побочной диагонали матрицы.
2.	Найти сумму элементов второй строки матрицы.
3.	Найти сумму элементов второго столбца матрицы.
4.	Вывести на печать элементы матрицы $2 \leq A[i, j] \leq 5$.
5.	Вывести на печать отрицательные элементы матрицы.
6.	Вывести на печать положительные элементы матрицы.
7.	Вычислить сумму элементов матрицы.
8.	Найти сумму элементов побочной диагонали матрицы.
9.	Вычислить сумму элементов двух первых столбцов матрицы.
10.	Вычислить среднее арифметическое матрицы.

Пример. Ввести квадратную матрицу X размером $R \times C$ с помощью функции генерирования случайных чисел (встроенного модуля `random`), допустим ввод с клавиатуры. При возникновении вопросов обратиться к лекционному материалу.

Представлена блок-схема решения задачи по нахождению суммы последней строки матрицы размером $R \times C$.

Пример решения задачи в виде блоксхемы



Листинг программы

```
import random # Модуль для генерации случайных чисел

def main():
    # Ввод количества строк
    R = int(input("Введите количество строк R: "))

    # Ввод количества столбцов
    C = int(input("Введите количество столбцов C: "))

    matrix = [] # Пустой список для будущей матрицы

    print("\nСгенерированная матрица:")

    # Заполнение матрицы случайными числами
    for i in range(R): # Цикл по строкам
        row = [] # Пустая строка
        for j in range(C): # Цикл по столбцам
            row.append(random.randint(1, 100)) # Добавляем случайное число от 1 до 100
        matrix.append(row) # Добавляем строку в матрицу
        print(row) # Вывод строки

if __name__ == "__main__":
    main() # Запуск программы
```

Результат программы

```
Введите количество строк R: 2
Введите количество столбцов C: 4

Сгенерированная матрица:
[21, 35, 18, 88]
[71, 55, 17, 62]
```

Ваша задача выполнить задание по варианту, при
возникновении вопросов обратиться к лекционному материалу.

Дополнительное задание: Самостоятельно создать задачу для
нахождения данных в массиве и решить её.

Методические указания

1. Определение размеров массива

В начале работы следует получить числовые параметры, определяющие форму двумерной структуры (можно использовать ввод пользователя).

2. Создание структуры для хранения данных

Вспомните, что в Python нет встроенного типа «матрица», поэтому её

обычно формируют из вложенных коллекций. Важно учитывать, что каждая строка представляет собой самостоятельный объект.

3. Генерация содержимого

Для заполнения используется инструмент из стандартного набора Python. Обратите внимание, что диапазон и тип значений определяются в задании.

4. Организация двойного обхода

Структура двумерных данных предполагает вложенный цикл. Один отвечает за строки, другой — за элементы в строке. Подумайте, какие индексы за что отвечают.

5. Вывод данных

Достаточно выполнить отображение по мере формирования или использовать итоговый вывод после построения всей структуры.

6. Логика основной задачи

В зависимости от варианта задания необходимо выполнить действие с данными массива.

Требования к отчету

- Титульный лист
- Задание.
- Блок схему
- Листинг программы
- Результат выполнения программы.

Контрольные вопросы

1. Отличие задания одномерного массива от двумерного в Python.
2. Как создать двумерный массив случайных целых чисел?
3. Как создать двумерный массив вещественных чисел?
4. Какая инструкция предусмотрена для выполнения итерации элементов массиве?
5. Каков формат объявления двумерного массива?
6. С чего начинается индексация массивов?

Лабораторная работа № 9 — Подпрограммы

Предмет исследований: Правила объявления подпрограмм. Связь формальных и фактических параметров. Способы передачи фактических параметров в подпрограмму. Обращение к функциям.

Задание

Создать программу, использующую 2 подпрограммы- функции. Числа должны вводиться с клавиатуры. Формат числа вещественный, дробная часть вводимого числа не менее 5 знаков.

Первая функция должна возвращать результат произведения чисел. Вторая функция должна возвращать результат деления двух чисел. При выводе числа в дробной части отображать только 2 знака.

Пример

(Обратите внимание, что пример для целых чисел, а в задании для вещественных. При этом в примере 3 разных числа.)

Создать программу, использующую 2 подпрограммы-функции:

- Первая функция должна возвращать наибольший общий делитель NOD двух целых чисел.
- Вторая функция должна возвращать наименьшее общее делимое NOK двух целых чисел.

Обе функции в главной программе должны использоваться с тремя разными целыми числами, вводимыми с клавиатуры. Проект – консольное приложение.

Листинг программы:

```
def nod(x, y):
    """Функция определения наибольшего общего делителя (НОД)"""
    if x != 0:
        return nod(y % x, x) # Рекурсивный вызов
    else:
        return y

def nok(x, y):
    """Функция определения наименьшего общего кратного (НОК)"""
    return (x // nod(x, y)) * y

def main():
    print("Введите через Enter 3 целых числа a b c:")
    a = int(input())
    b = int(input())
    c = int(input())
    print()

    print("Наименьшие общие кратные (НОК) двух целых чисел:")
    print(f"NOK({a}, {b}) = {nok(a, b)}")
    print(f"NOK({a}, {c}) = {nok(a, c)}")
    print(f"NOK({b}, {c}) = {nok(b, c)}")
    print()

    print("Наибольшие общие делители (НОД) двух целых чисел:")
    print(f"NOD({a}, {b}) = {nod(a, b)}")
    print(f"NOD({a}, {c}) = {nod(a, c)}")
    print(f"NOD({b}, {c}) = {nod(b, c)}")

    input("\nНажмите Enter для выхода...")

if __name__ == "__main__":
    main()
```

```
Введите через Enter 3 целых числа a b c:
3
4
2

Наименьшие общие кратные (НОК) двух целых чисел:
NOK(3, 4) = 12
NOK(3, 2) = 6
NOK(4, 2) = 4

Наибольшие общие делители (НОД) двух целых чисел:
NOD(3, 4) = 1
NOD(3, 2) = 1
NOD(4, 2) = 2
```

Рис 1.1 – Результат программы

Методические указания

1. Создание функций

Реализуйте первую функцию для умножения двух чисел и вторую функцию для деления двух чисел. Убедитесь, что функции принимают параметры и возвращают результаты вычислений.

2. Организация ввода данных

Запросите ввод трех вещественных чисел с клавиатуры. Убедитесь, что числа имеют дробную часть не менее 5 знаков (например: 12.34567, 7.89012, 45.67890).

3. Вычисление результатов для всех пар

Используйте созданные функции для вычисления произведения и частного для всех возможных пар чисел: (a,b), (a,c) и (b,c). Сохраните результаты вычислений.

4. Форматирование вывода результатов

Организируйте вывод результатов с округлением до двух знаков после запятой. Используйте форматированный вывод для улучшения читаемости результатов.

5. Обработка особых случаев

Продумайте обработку ситуации деления на ноль. Реализуйте проверку делителя перед выполнением операции деления.

6. Тестирование программы

Проверьте работу программы на различных тестовых данных. Убедитесь в корректности вычислений и форматирования вывода для всех пар чисел.

Требования к отчёту

Отчёт подготавливается в электронном виде. Он должен содержать постановку задачи, скриншоты, которые показывают порядок решения задачи, необходимые комментарии и анализ полученных результатов. И электронное решение задач.

Контрольные вопросы:

1. Что такое подпрограмма? Её назначение.
2. Правила объявления функций в программе.
3. Правила обращения к функции в программе.
4. Формальные и фактические параметры.

Лабораторная 10 — Многооконные приложения

Предмет исследований

Изучение способов работы с несколькими окнами графического интерфейса в одном приложении на Python с использованием библиотеки **tkinter**:

- открытие и закрытие окон;
- организация парольного доступа к окнам;
- вывод сообщений об ошибках с помощью диалоговых окон (messagebox);
- вставка изображения в окно.

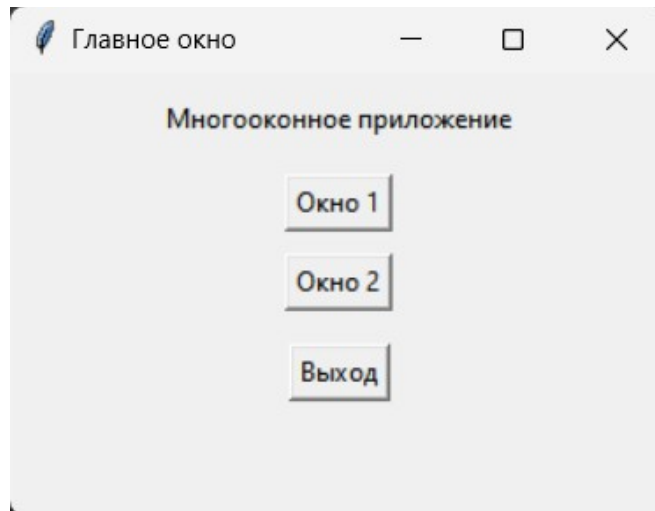
Задание.

Создать программу, в которой реализовано:

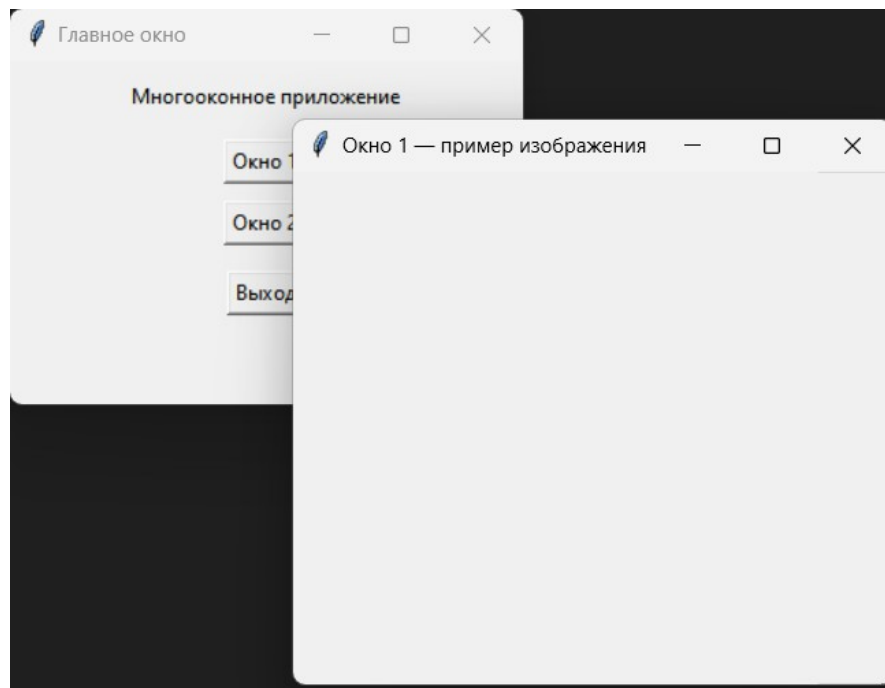
1. Главное окно с двумя кнопками — «Окно 1» и «Окно 2».
2. При нажатии кнопки «Окно 1» открывается второе окно с изображением.
3. При нажатии кнопки «Окно 2» появляется окно ввода пароля.
 - Если пароль введен правильно → открывается «Окно 4».
 - Если пароль неверный → выводится сообщение об ошибке с помощью `messagebox.showerror()`.
4. Программа завершается после закрытия главного окна.

Пример. В примере показана программа, демонстрирующая работу с многооконными приложениями и организацию парольного доступа к окнам, а также использование диалоговых окон **messagebox** для вывода сообщений. В программе используются компоненты **Button**, **Entry** и **Label**. Все эти элементы доступны в стандартной библиотеке **tkinter**, предназначенной для создания графических интерфейсов на языке Python.

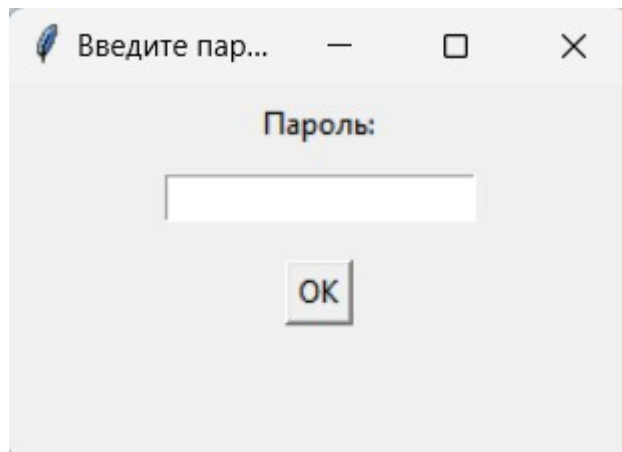
При запуске программы открывается "Главное окно" (форма 1) с двумя кнопками.



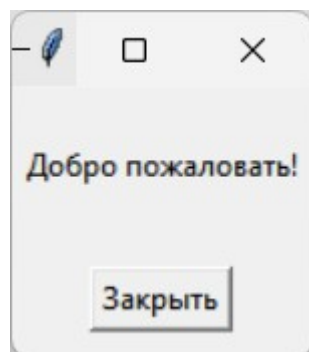
При нажатии на кнопку с именем "Окно 1" открывается "Окно 1" (форма 2) с картинкой. Например



Если пользователь нажимает кнопку "Окно 2", то вызывается форма 4, окно которой не отображается, а из нее вызывается форма 3. Появляется "Окно 3" (форма 3) с запросом пароля.



Если пароль введен правильно, то пользователь получает доступ к “Окну 4” (форма 4).



Окно с запросом пароля автоматически закрывается. Если пользователь закрывает окно ввода пароля, то приложение завершается.


```
import tkinter as tk
from tkinter import messagebox

# функция открытия окна 1
def open_window1():
    # создаём новое окно
    win1 = tk.Toplevel(root)
    win1.title("Окно 1 – пример изображения")
    win1.geometry("300x300")

    # загружаем изображение в формате PNG или GIF
    photo = tk.PhotoImage(file="photo.png")

    # создаём виджет Label и вставляем изображение
    label = tk.Label(win1, image=photo)
    label.image = photo # сохраняем ссылку, чтобы изображение не исчезло
    label.pack(pady=20)

    # кнопка закрытия окна
    tk.Button(win1, text="Закрыть", command=win1.destroy).pack(pady=10)

# функция для открытия окна 2 (ввод пароля)
def open_password_window():
    win2 = tk.Toplevel(root)
    win2.title("Введите пароль")
    win2.geometry("250x150")

    tk.Label(win2, text="Пароль:").pack(pady=5)
    password_entry = tk.Entry(win2, show="*")
    password_entry.pack(pady=5)

    def check_password():
        if password_entry.get() == "12345":
            win2.destroy()
            open_secret_window()
        else:
            messagebox.showerror("Ошибка", "Неверный пароль!")

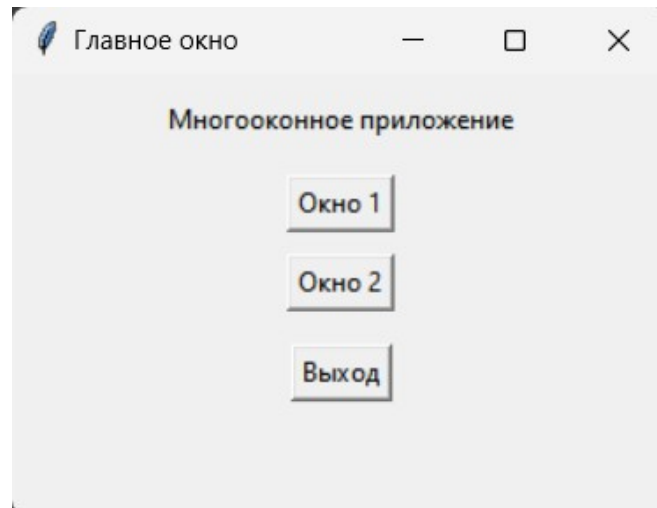
    tk.Button(win2, text="OK", command=check_password).pack(pady=10)

# функция для окна с секретным доступом
def open_secret_window():
    win3 = tk.Toplevel(root)
    win3.title("Окно 4 – Доступ разрешён")
    tk.Label(win3, text="Добро пожаловать!").pack(pady=20)
    tk.Button(win3, text="Закрыть", command=win3.destroy).pack(pady=10)

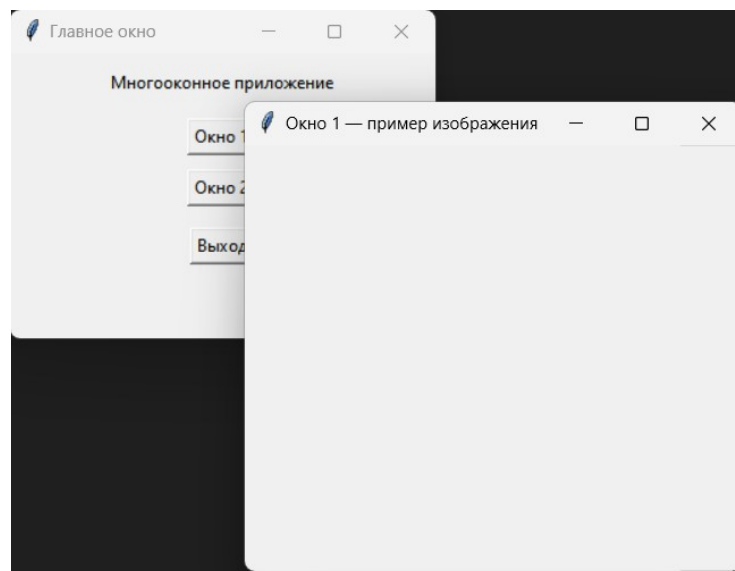
tk.Label(root, text="Многооконное приложение").pack(pady=10)
tk.Button(root, text="Окно 1", command=open_window1).pack(pady=5)
tk.Button(root, text="Окно 2", command=open_password_window).pack(pady=5)
tk.Button(root, text="Выход", command=root.destroy).pack(pady=10)

root.mainloop()
```

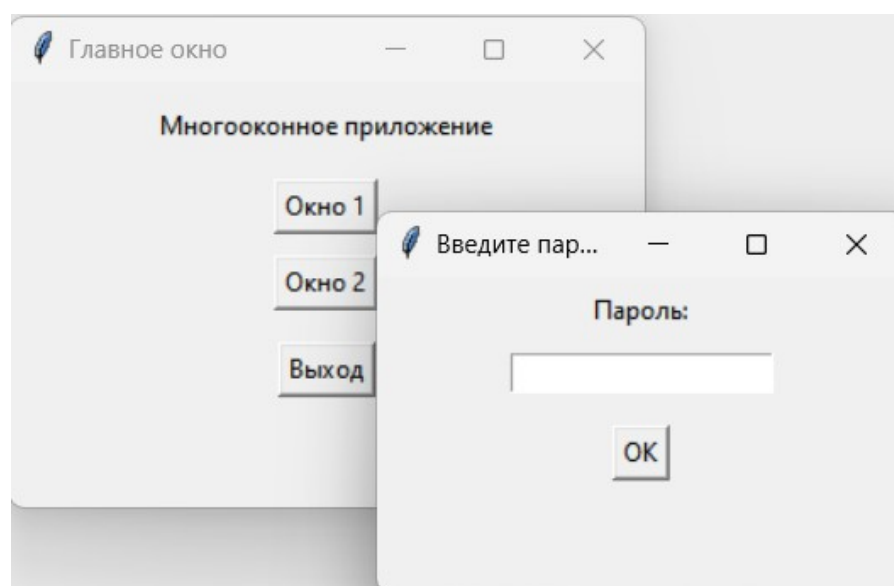
При запуске программы мы видим следующее:



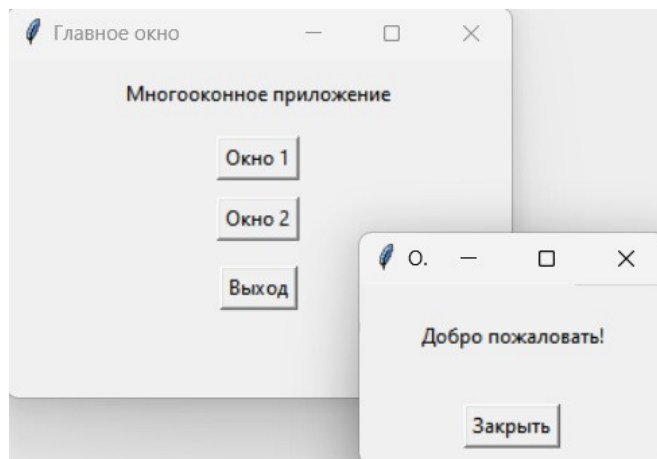
Если нажата кнопка «Окно 1», то



Если нажата кнопка «Окно 2», то



Если пароль правильный, то



Методические указания

1. Создание главного окна

Реализуйте главное окно приложения с использованием класса Tk(). Настройте заголовок окна и его размеры. Убедитесь, что окно корректно отображается и имеет две кнопки "Окно 1" и "Окно 2".

```
import tkinter as tk
# Создание главного окна
root = tk.Tk()
root.title("Главное окно")
root.geometry("300x200")
```

2. Реализация функции открытия окна с изображением

Создайте функцию, которая будет открывать новое окно с изображением при нажатии кнопки "Окно 1". Используйте класс Toplevel() для создания дочернего окна и PhotoImage для загрузки изображения.

```
def open_image_window():
    # Создание дочернего окна
    image_window = tk.Toplevel(root)
    image_window.title("Окно с изображением")

    # Загрузка и отображение изображения
    try:
        image = tk.PhotoImage(file="image.png")
        label = tk.Label(image_window, image=image)
        label.image = image # Сохранение ссылки на изображение
        label.pack()
    except:
        # Обработка ошибки загрузки изображения
        error_label = tk.Label(image_window, text="Изображение не найдено")
        error_label.pack()
```

3. Организация парольного доступа

Реализуйте систему проверки пароля при нажатии кнопки "Окно 2". Создайте окно ввода пароля с использованием виджета Entry с параметром show="*". Организуйте проверку введенного пароля.

```
def open_password_window():
    # Создание окна ввода пароля
    password_window = tk.Toplevel(root)
    password_window.title("Ввод пароля")
    password_window.geometry("250x150")

    def check_password():
        entered_password = password_entry.get()
        if entered_password == "12345": # Проверка пароля
            password_window.destroy()
            open_protected_window()
        else:
            # Вывод сообщения об ошибке
            tk.messagebox.showerror("Ошибка", "Неверный пароль!")

    # Элементы окна ввода пароля
    tk.Label(password_window, text="Введите пароль:").pack(pady=10)
    password_entry = tk.Entry(password_window, show="*")
    password_entry.pack(pady=5)
    tk.Button(password_window, text="Войти", command=check_password).pack(pady=10)
```

4. Создание защищенного окна

Реализуйте функцию открытия окна, доступного только после успешного ввода пароля. Настройте внешний вид защищенного окна и добавьте информационное сообщение.

```
def open_protected_window():
    protected_window = tk.Toplevel(root)
    protected_window.title("Защищенное окно")
    protected_window.geometry("300x200")

    # Добавление содержимого в защищенное окно
    tk.Label(protected_window, text="Доступ разрешен!",
             font=("Arial", 14), fg="green").pack(pady=50)
    tk.Label(protected_window, text="Это защищенное окно").pack()
```

5. Организация элементов главного окна

Разместите кнопки в главном окне с использованием менеджера геометрии pack(). Настройте отступы между кнопками для улучшения визуального восприятия.

```
# Создание кнопок в главном окне
btn_window1 = tk.Button(root, text="Окно 1", command=open_image_window)
btn_window1.pack(pady=20)

btn_window2 = tk.Button(root, text="Окно 2", command=open_password_window)
btn_window2.pack(pady=20)
```

6. Обработка особых случаев

Продумайте обработку возможных ошибок:

- Ошибка загрузки изображения
- Неверный ввод пароля
- Непредвиденное закрытие окон

Реализуйте соответствующие сообщения об ошибках с использованием messagebox.

7. Завершение работы приложения

Обеспечьте корректное завершение работы приложения при закрытии главного окна. Убедитесь, что все дочерние окна закрываются вместе с главным.

```
# Обработка закрытия главного окна  
root.protocol("WM_DELETE_WINDOW", root.quit)
```

Контрольные вопросы

1. Открытие новых окон в приложениях.
2. Организация доступа к окнам.
3. Работа с MessageBox.
4. Организация завершения приложения.
5. Интеграция изображений в форму.
6. Закрытие окно в приложении.
7. Работа с событием FormClosing.
8. Объекты Show или Showdialog.